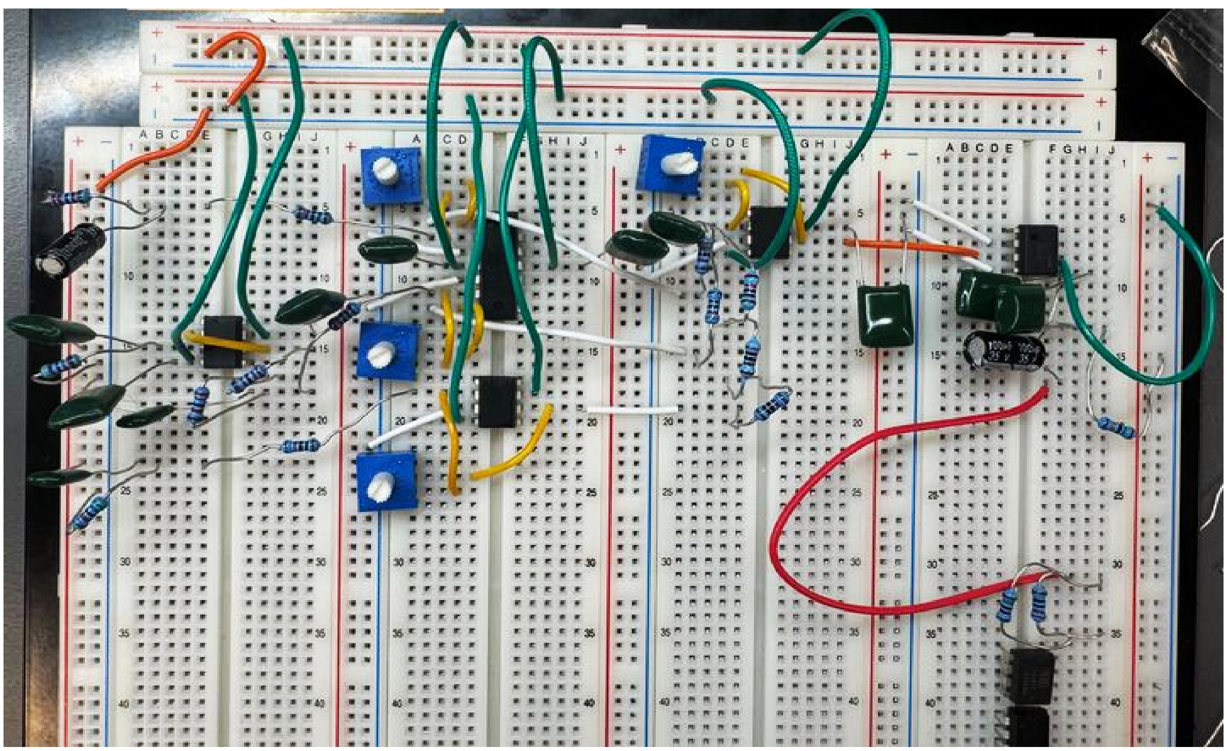
# Audio Equalizer

## Introduction

### Abstract

Audio equalizers are used in the music industry to tame feedback and ensure instrument sounds and vocal sounds occupy their own clear frequency space. In studio mixing, they sculpt individual tracks by boosting warmth in bass guitars, adding presence to vocals, and carving out mud in mid-range instruments to create a cohesive mix. In personal audio devices, EQs compensate for room acoustics or listener preferences, letting you boost the bass and soften the highs. This audio equalizer has a bass gain control, a mid gain control, a treble gain control, and a general volume control for the user to customize their sound. The audio equalizer will read in a signal and send the signal to a low-pass filter (bass filter), a band-pass filter (mid filter), and a high-pass filter (treble filter). The filters will output a signal, dependent on their respective gain controls, and get recombined. An output will pass through a power amplifier before passing into the speaker.



Check out a demonstration here: [Audio Equalizer Project Demo]

## Theory

### Filter Theory

Low-pass filters work by letting signals with low frequencies pass, high-pass filters work by letting signals with high frequencies pass, and band-pass filters work by letting a certain range of signals with various frequencies pass. This "passing point" is determined by a -3dB cutoff, also known as a cutoff frequency.

Schematics and equations for high-pass, low-pass, and band-pass filters can be found below. As shown, the calculation for the cutoff frequency for a high-pass and low-pass filter is the same. What determines whether or not the circuit is a high-pass or low-pass filter is dependent on which component voltage is being read; for a low-pass filter, the voltage is read across the capacitor, and for a high-pass filter, the voltage is read across the resistor. The band-pass filter is a high-pass filter with the output sent to a low-pass filter, with a buffer to prevent loading issues.

Image I: Schematics for low-pass filter, high-pass filter, and band-pass filter.

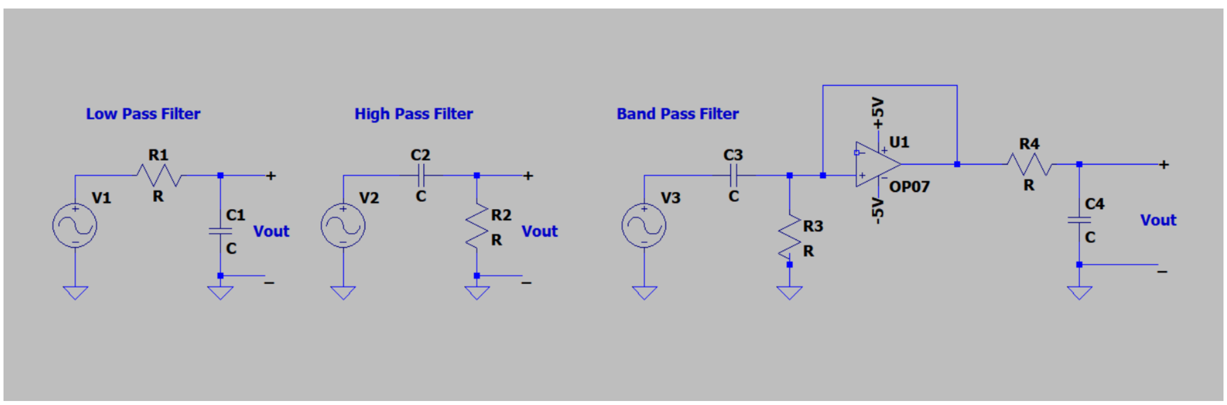



Table I: Equations to calculate cutoff frequencies for low-pass filter, high-pass filter, and band-pass filter.

| Low Pass Filter | High Pass Filter | Band Pass Filter |
|---|---|---|
| $f_c = 1/(2\pi RC)$ | $f_c = 1/(2\pi RC)$ | $f_{c\,low} = (-R/2L + \sqrt{(R/2L)^2 + 1/LC}\ ) / 2\pi$ <br> $f_{c\,high} = (R/2L + \sqrt{(R/2L)^2 + 1/LC}\ ) / 2\pi$ |

**Operational Amplifier Theory**

An operational amplifier, or op amp, is an amplifier that amplifies the voltage difference between its inverting and non-inverting inputs. Op amps are widely used in analog circuits and can be configured to do various things by adding external feedback networks. Some examples of these configurations include gain control, buffering, active filtering, and mathematical operations (summing, differentiating, and integrating). In this audio equalizer, op amps are used for buffering, summing, and gain control.

Schematics and equations for the various configurations of the op amp can be found below. For the gain control op amps and the summing op amp , potentiometers are used to adjust the value of the feedback resistance, allowing the user to adjust the gain of that signal.

Image II: Schematics for various op amp configurations used in the audio equalizer

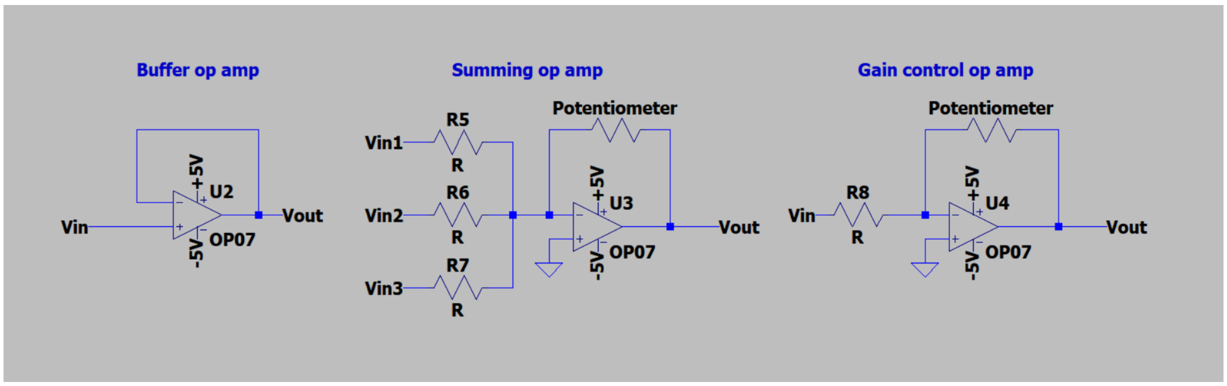



Table II: Equations for Vout of each op amp configuration used in the audio equalizer

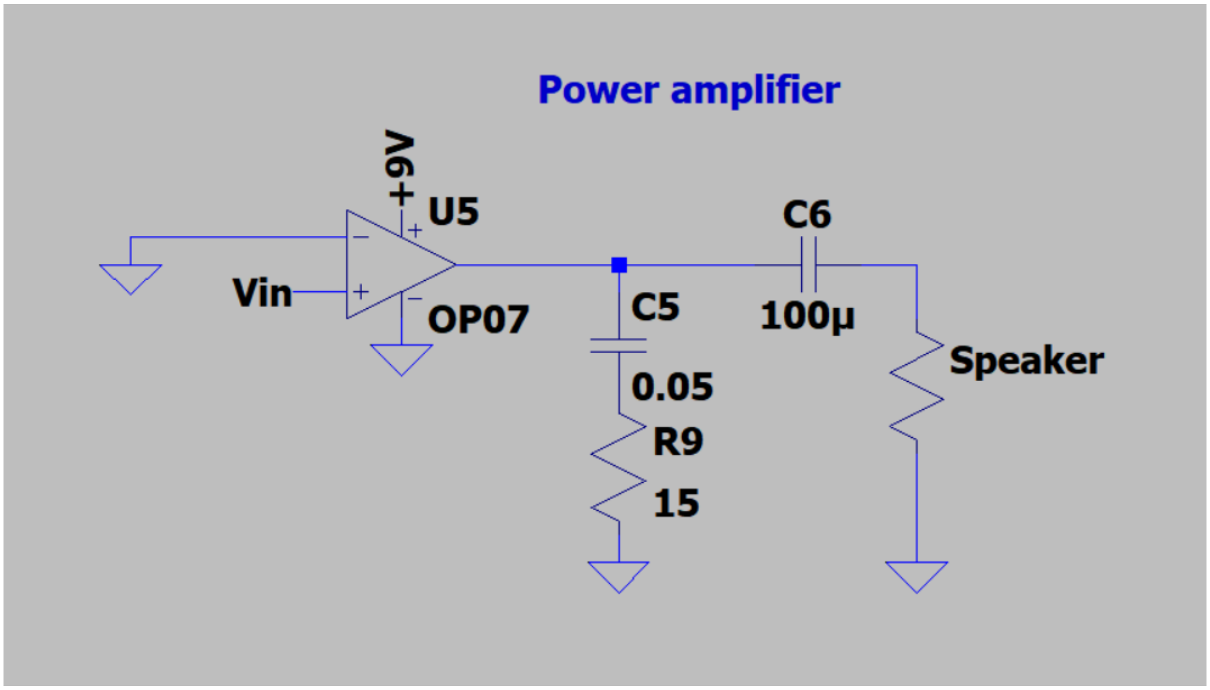
| Buffer op amp | Summing op amp | Gain control op amp |
|---|---|---|
| n/a | $V_{out} = -R_{pot}(V_{in1}/R_5 + V_{in2}/R_6 + V_{in3}/R_7)$ | $V_{out} = V_{in}(R_{pot}/R_8)$ |

**Power Amplifier Theory**

Power amplifiers take low-level input signals and boost their power to deliver substantial values to loads such as loudspeakers or RF antennas. They preserve signal fidelity by controlling distortion and stability while driving demanding output stages. In this audio equalizer, a power amplifier is used to boost the output signal of the summing op amp to send to the speaker.

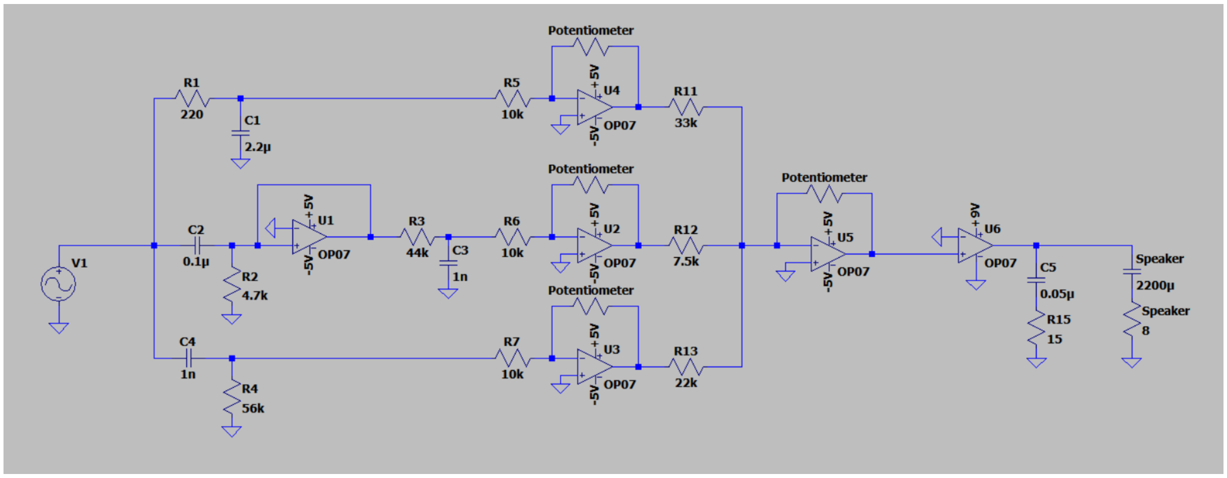
The schematic for the power amplifier can be found below.

Image III: Schematic for power amplifier



## Procedure

Image IV: Schematic of audio equalizer.




### Filter Design Parameters

Using the equations from the Filter Theory section, I derived the necessary cutoff frequencies for each of my filters. This included a 320Hz cutoff for the low pass filter, a 3200Hz cutoff for the high pass filter, and a 320Hz to 3200Hz band pass for the band pass filter.

I started by writing a Python script that would calculate the optimal resistor and capacitor values for each filter. These scripts are attached below. The low-pass and high-pass filters use the same script because their cutoff frequencies are calculated in the same manner. The band-pass filter should theoretically use the same values as the low and high-pass filters, except the components between the two filters should be swapped because the cutoff frequencies are also swapped. The high-pass filter values I chose were not the most optimal, because I wanted high resistor values to fix loading issues. The low-pass filter was allowed to have a low resistor value because the filter reads the voltage across the capacitor and not the resistor. Another reason why I didn't choose the most optimal values was because of the parts. I only had a limited number of capacitors, so I scaled the capacitors down and the resistors up. The results of the scripts (the values I chose initially) are shown in Table III below.

Once these values were calculated, I built the filters on a breadboard. Due to oscilloscope impedances and frequency response analysis (FRA) inaccuracies, I tweaked the resistor and capacitor values of the filters until each filter showed a proper response. The final resistor and capacitor values are shown in Table IV below, and the audio equalizer schematic.

Image V: Python script for calculating component values for high-pass and low-pass filters

```python
from math import pi

def main():
    caps = [0.001, 0.01, 0.47, 1, 2.2, 4.7, 10, 22, 100]
    resistors = [10, 100, 1000, 10000, 100000, 1000000, 10000000, 127000, 15, 150, 1500,
                 15000, 150000, 22, 220, 2200, 22000, 200000, 220000, 33, 330, 3300, 33000,
                 330000, 47, 470, 4700, 47000, 470000, 56, 560, 5600, 56000, 560000, 68,
                 680, 6800, 68000, 680000, 82, 820, 8200, 82000, 820000] * 10

    cutoffFrequency = input("Enter the cutoff frequency in Hz: ")
    try:
        cutoffFrequency = float(cutoffFrequency)
    except ValueError:
        print("Invalid input. Please enter a numeric value for the cutoff frequency.")
        return

    closest_combination = None
    closest_difference = float('inf')

    for cap in caps:
        cap *= 10**-6
        for res in resistors:
            frequency = 1 / (2 * pi * cap * res)
            difference = abs(frequency - cutoffFrequency)
            if difference < closest_difference:
                closest_difference = difference
                closest_combination = (cap, res)
                print(f"→ New best: C = {cap:.3e} F, R = {res} Ω gives f = {frequency:.1f} Hz (Δ = {difference:.1f})")

    if closest_combination:
        print(f"Closest combination: Capacitor = {closest_combination[0] / (10**-6)} µF, Resistor = {closest_combination[1]} Ω")
        print(f"Resulting frequency: {1 / (2 * pi * closest_combination[0] * closest_combination[1]):.2f} Hz")
        print(f"Difference from cutoff frequency: {closest_difference:.2f} Hz")

if __name__ == "__main__":
    main()
```

Image VI: Python script for calculating component values for the band-pass filter

```python
from math import pi
from itertools import combinations

def calculate_series_resistance(r1, r2):
    return r1 + r2

def calculate_parallel_resistance(r1, r2):
    return 1 / (1 / r1 + 1 / r2)

def main():
    caps = [0.001, 0.01, 0.47, 1, 2.2, 4.7, 10, 22, 100]
    resistors = [10, 100, 1000, 10000, 100000, 1000000, 10000000, 127000, 15, 150, 1500,
                 15000, 150000, 22, 220, 2200, 22000, 200000, 220000, 33, 330, 3300,
                 33000, 330000, 47, 470, 4700, 47000, 470000, 56, 560, 5600, 56000, 560000,
                 68, 680, 6800, 68000, 680000, 82, 820, 8200, 82000, 820000] * 10

    # Generate all possible resistor combinations
    for r1, r2 in combinations(resistors, 2):
        resistors.append(calculate_series_resistance(r1, r2))
        resistors.append(calculate_parallel_resistance(r1, r2))

    # Remove duplicates and sort the resistor list
    resistors = sorted(set(resistors))

    cutoffFrequencylow = int(input("Enter the low cutoff frequency in Hz: "))
    cutoffFrequencyhigh = int(input("Enter the high cutoff frequency in Hz: "))

    closest_combination = None
    closest_difference = float('inf')

    ind = 0.001 # inductor value in H
```

```python
    for cap in caps:
        cap *= 10**-6    # now cap is in farads
        for res in resistors:

            # compute the two band-pass poles
            frequency1 = (-res/(2*ind)+ ((res/(2*ind))**2 + 1/(ind*cap))**0.5)/(2*pi)
            frequency2 = (res/(2*ind)+((res/(2*ind))**2 + 1/(ind*cap))**0.5)/(2*pi)

            # score by _both_ errors
            difference = (abs(frequency1 - cutoffFrequencylow) + abs(frequency2 - cutoffFrequencyhigh))

            if difference < closest_difference:
                closest_difference  = difference
                # store cap (in F), res, and both freqs
                closest_combination = (cap, res, frequency1, frequency2)
                print(
                    f"→ New best: C={cap:.3e} F, "
                    f"R={res} Ω → "
                    f"f_low={frequency1:.1f}Hz, "
                    f"f_high={frequency2:.1f}Hz "
                    f"(Δ={difference:.1f})"
                )

    if closest_combination:
        cap, res, f1, f2 = closest_combination
        # convert back to μF for reporting
        print("\nBest overall match:")
        print(f"  C = {cap/(10**-6)} μF, R = {res:.2f} Ω")
        print(f"  → f_low  = {f1:.2f} Hz (target {cutoffFrequencylow} Hz)")
        print(f"  → f_high = {f2:.2f} Hz (target {cutoffFrequencyhigh} Hz)")
        print(f"Total deviation = {closest_difference:.2f} Hz")

if __name__ == "__main__":
    main()
```

Image VII: Result of script (low-pass filter)

```
Enter the cutoff frequency in Hz: 320
→ New best: C = 1.000e-09 F, R = 10 Ω gives f = 15915494.3 Hz (Δ = 15915174.3)
→ New best: C = 1.000e-09 F, R = 100 Ω gives f = 1591549.4 Hz (Δ = 1591229.4)
→ New best: C = 1.000e-09 F, R = 1000 Ω gives f = 159154.9 Hz (Δ = 158834.9)
→ New best: C = 1.000e-09 F, R = 10000 Ω gives f = 15915.5 Hz (Δ = 15595.5)
→ New best: C = 1.000e-09 F, R = 100000 Ω gives f = 1591.5 Hz (Δ = 1271.5)
→ New best: C = 1.000e-09 F, R = 1000000 Ω gives f = 159.2 Hz (Δ = 160.8)
→ New best: C = 1.000e-09 F, R = 470000 Ω gives f = 338.6 Hz (Δ = 18.6)
→ New best: C = 2.200e-06 F, R = 220 Ω gives f = 328.8 Hz (Δ = 8.8)
Closest combination: Capacitor = 2.2 μF, Resistor = 220 Ω
Resulting frequency: 328.83 Hz
Difference from cutoff frequency: 8.83 Hz
```

Image VIII: Result of script (high-pass filter)

```
Enter the cutoff frequency in Hz: 3200
→ New best: C = 1.000e-09 F, R = 10 Ω gives f = 15915494.3 Hz (Δ = 15912294.3)
→ New best: C = 1.000e-09 F, R = 100 Ω gives f = 1591549.4 Hz (Δ = 1588349.4)
→ New best: C = 1.000e-09 F, R = 1000 Ω gives f = 159154.9 Hz (Δ = 155954.9)
→ New best: C = 1.000e-09 F, R = 10000 Ω gives f = 15915.5 Hz (Δ = 12715.5)
→ New best: C = 1.000e-09 F, R = 100000 Ω gives f = 1591.5 Hz (Δ = 1608.5)
→ New best: C = 1.000e-09 F, R = 47000 Ω gives f = 3386.3 Hz (Δ = 186.3)
→ New best: C = 2.200e-06 F, R = 22 Ω gives f = 3288.3 Hz (Δ = 88.3)
Closest combination: Capacitor = 2.2 μF, Resistor = 22 Ω
Resulting frequency: 3288.33 Hz
Difference from cutoff frequency: 88.33 Hz
```

Table III: Selected component values based on the output of the script.

| | **High Pass** | **Low Pass** | **High Pass for Band Pass** | **Low Pass for Band Pass** |
|---|---|---|---|---|
| **Resistor (Ω)** | 47k | 220 | 4.7k | 47k |
| **Capacitor (F)** | 1n | 2.2µ | 0.1µ | 1n |

Table IV: Tweaked component values based on FRA results. (These values were the final ones I chose)

| | **High Pass** | **Low Pass** | **High Pass for Band Pass** | **Low Pass for Band Pass** |
|---|---|---|---|---|
| **Resistor (Ω)** | 56k | 220 | 4.7k | 44k |
| **Capacitor (F)** | 1n | 2.2µ | 0.1µ | 1n |

**Gain/Volume Control Design Parameters**

For this audio equalizer, I decided to make all the filter gain controls have a gain of 0 to 1. Since potentiometers theoretically vary from 0 ohms to 10k ohms of resistance, using the equations from the Operational Amplifier Theory section, I chose 10k resistors as loading resistors to get a max gain of 1.

The summing amplifier volume control is more complex. Theoretically, since the signal should only be passing through one of the filters depending on its frequency (the other filters are blocking the signal), the loading resistor for each respective filter input to the summing amplifier should be 33k based on the equation from the Operational Amplifier Theory section. However, since these filters are not ideal, there will be variance based on the performance of the filters. After testing and tweaking the filters over and over again, I came up with 33k ohms for the low pass input, 7.5k ohms for the band pass input, and 22k ohms for the high pass input.

All of these loading resistor values can be seen in the audio equalizer schematic.

## Results

### FRA Results

For these FRA curves, you can note the cutoff frequency at the point where -3dB is reached. These points are marked with the oscilloscope cursors on the image.

Image IX: Low-pass filter FRA results.

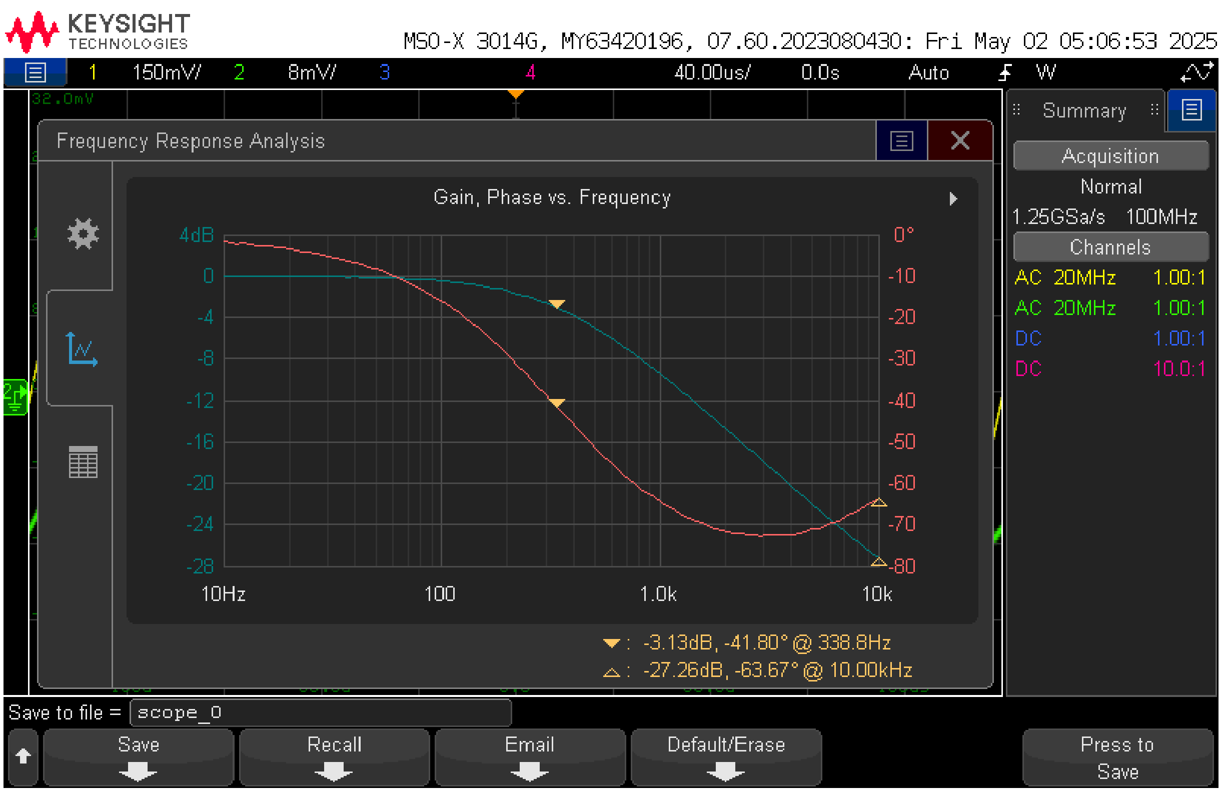


Image X: Band-pass filter FRA results.

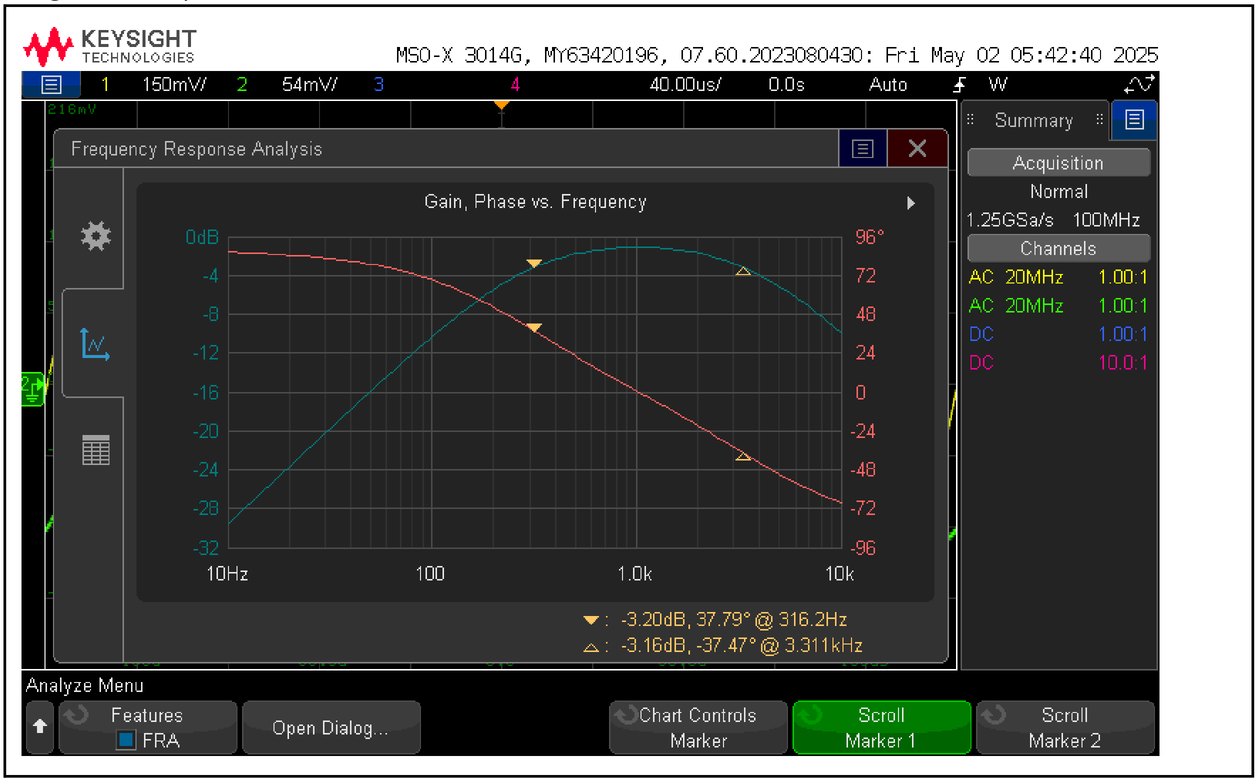
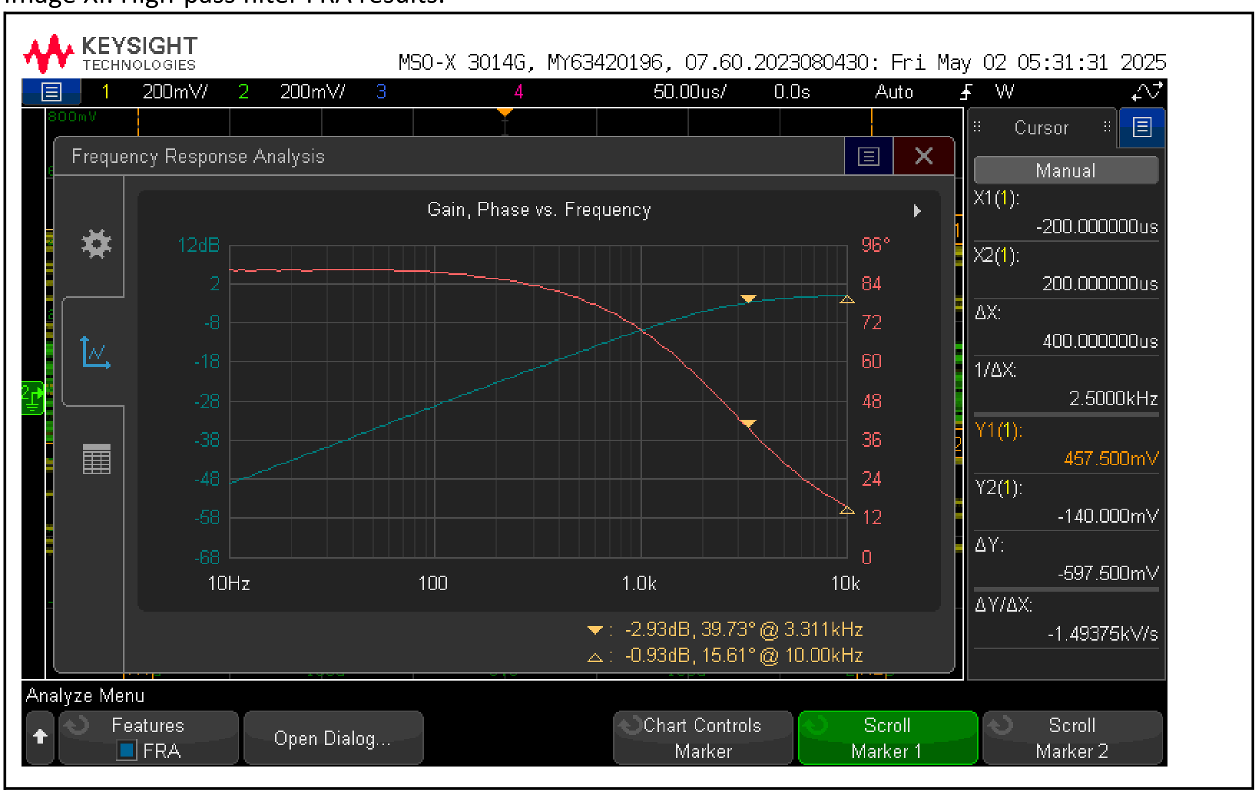



Image XI: High-pass filter FRA results.



**Output Results**

In the plots below, you can see the output of the circuit when the equalizer is turned to minimum and maximum settings. As seen in the plots, at minimum settings, the RMS voltage is less than 15mVRMS, and at maximum settings, the RMS voltage is 100mVRMS ±10%.

Image XII: Output voltage with equalizer turned to minimum settings at 100Hz

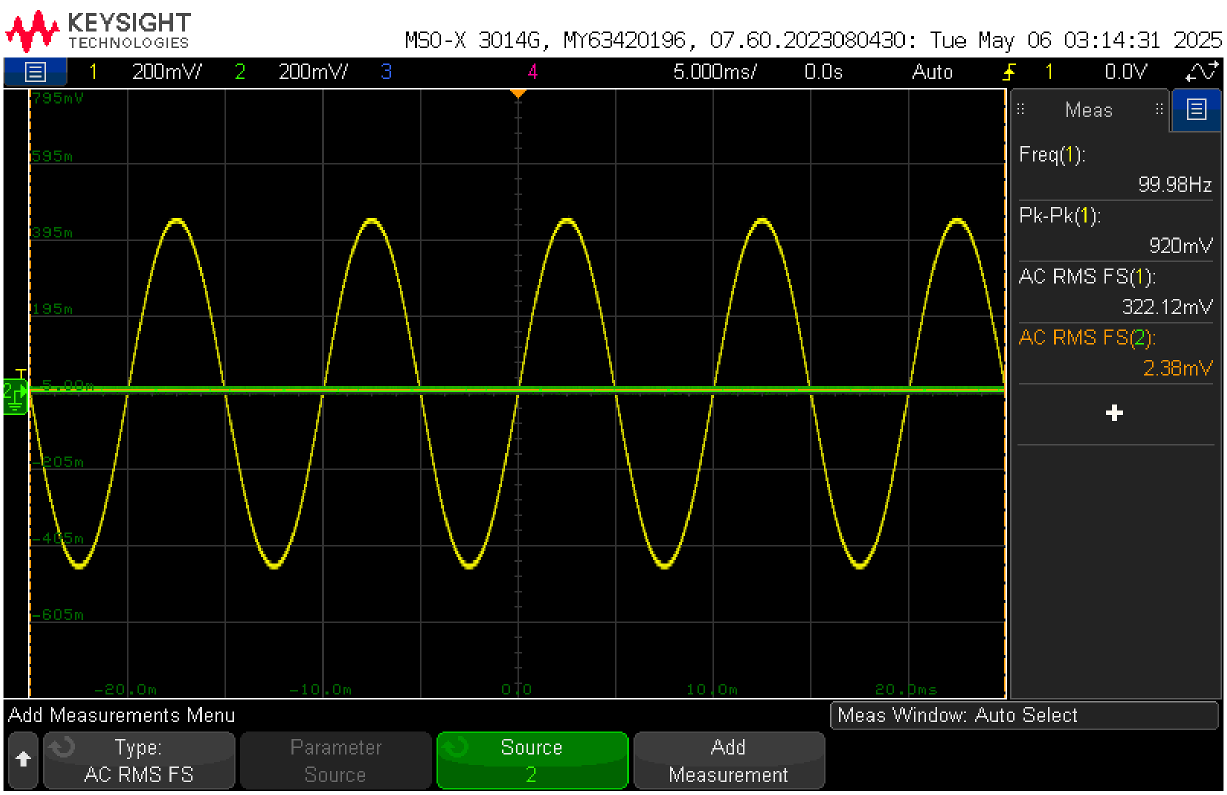


Image XIII: Output voltage with equalizer turned to minimum settings at 1kHz

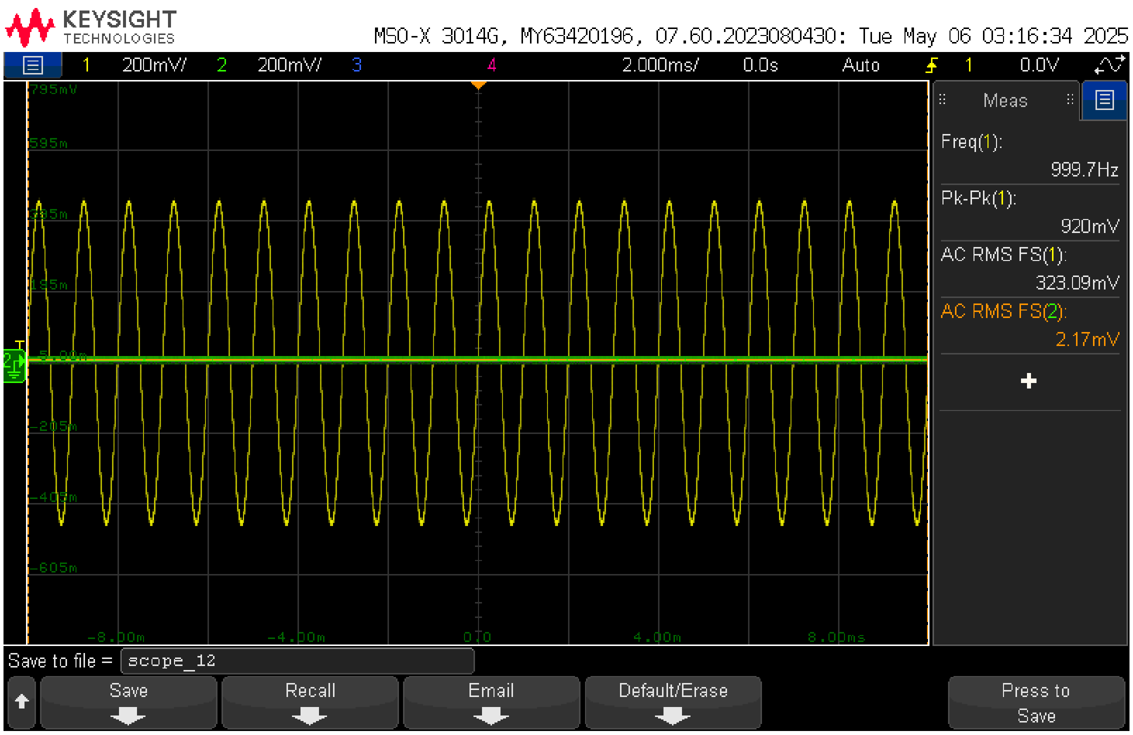



Image XIV: Output voltage with equalizer turned to minimum settings at 10kHz

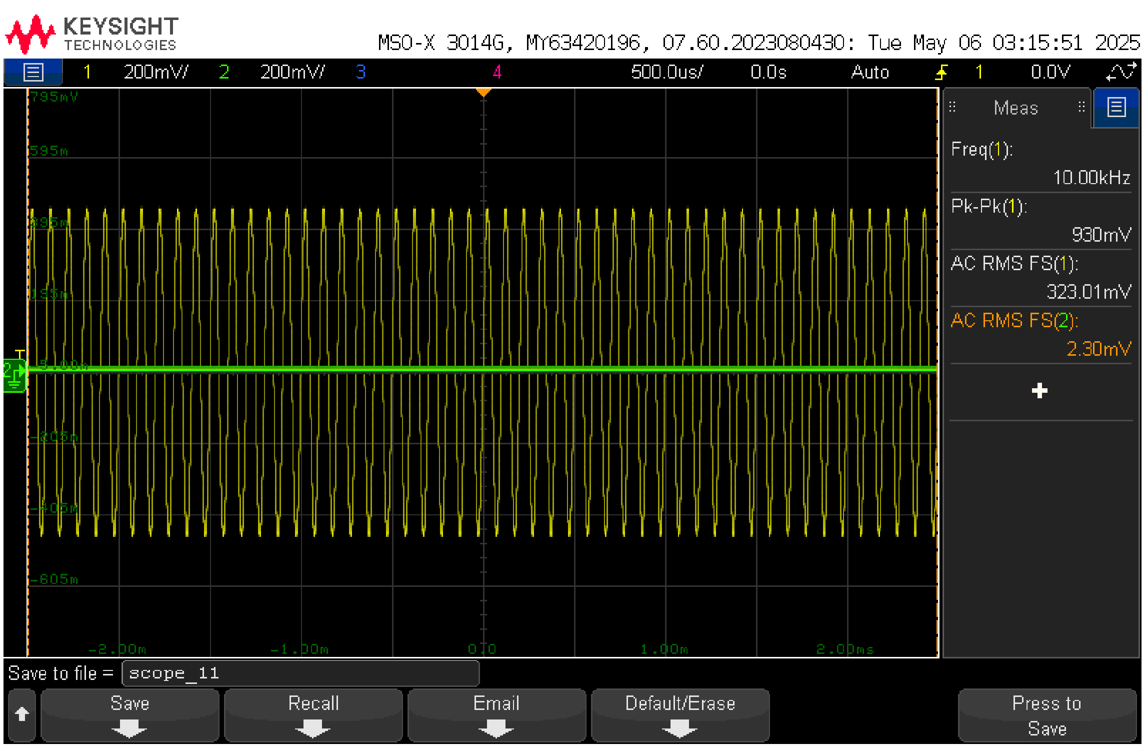


Image XV: Output voltage with equalizer turned to maximum settings at 100Hz

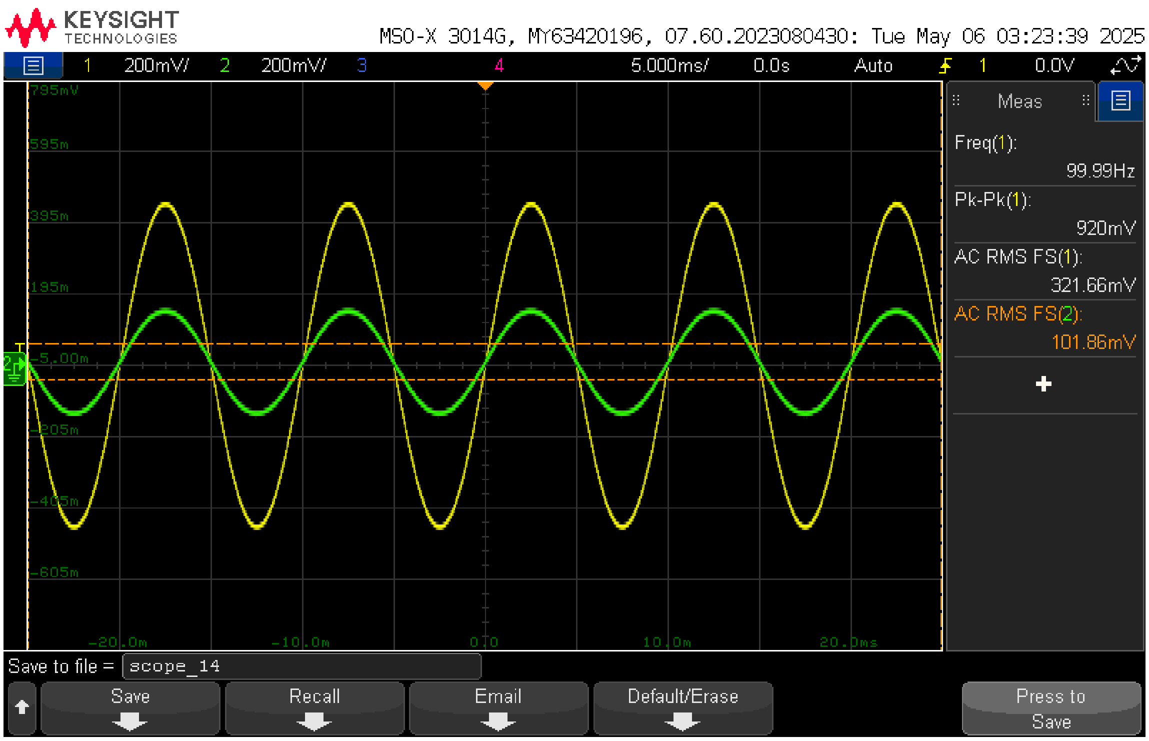



Image XVI: Output voltage with equalizer turned to maximum settings at 1kHz

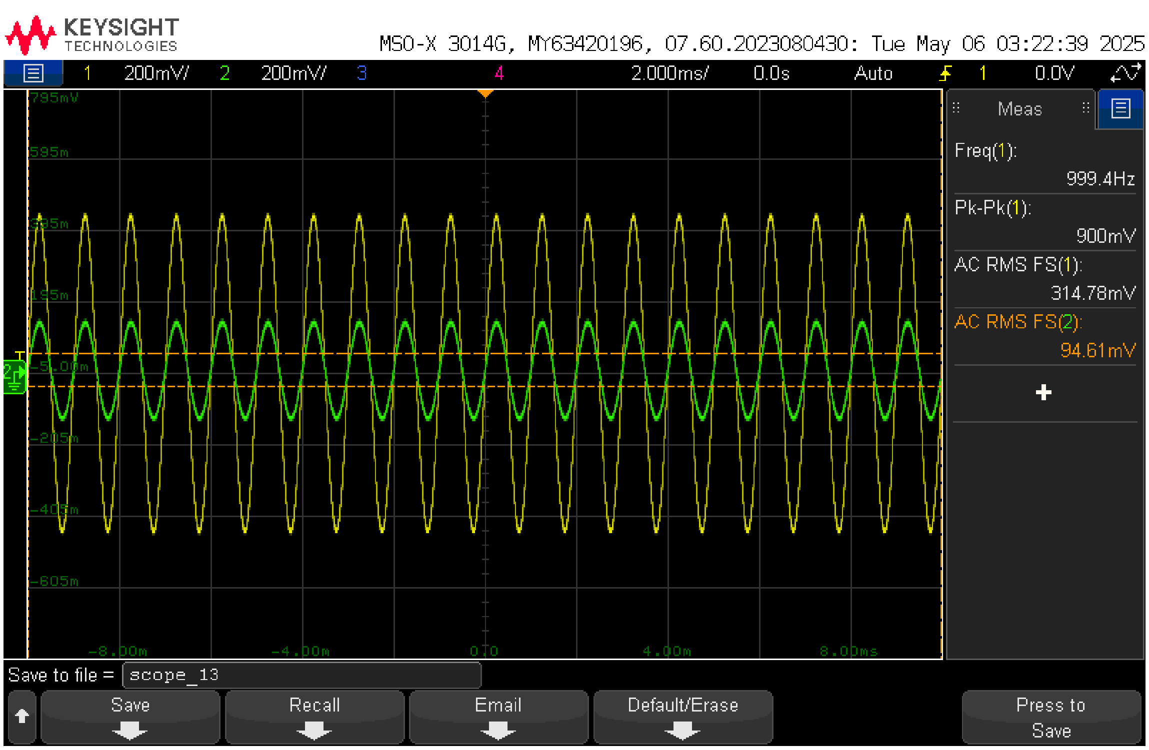
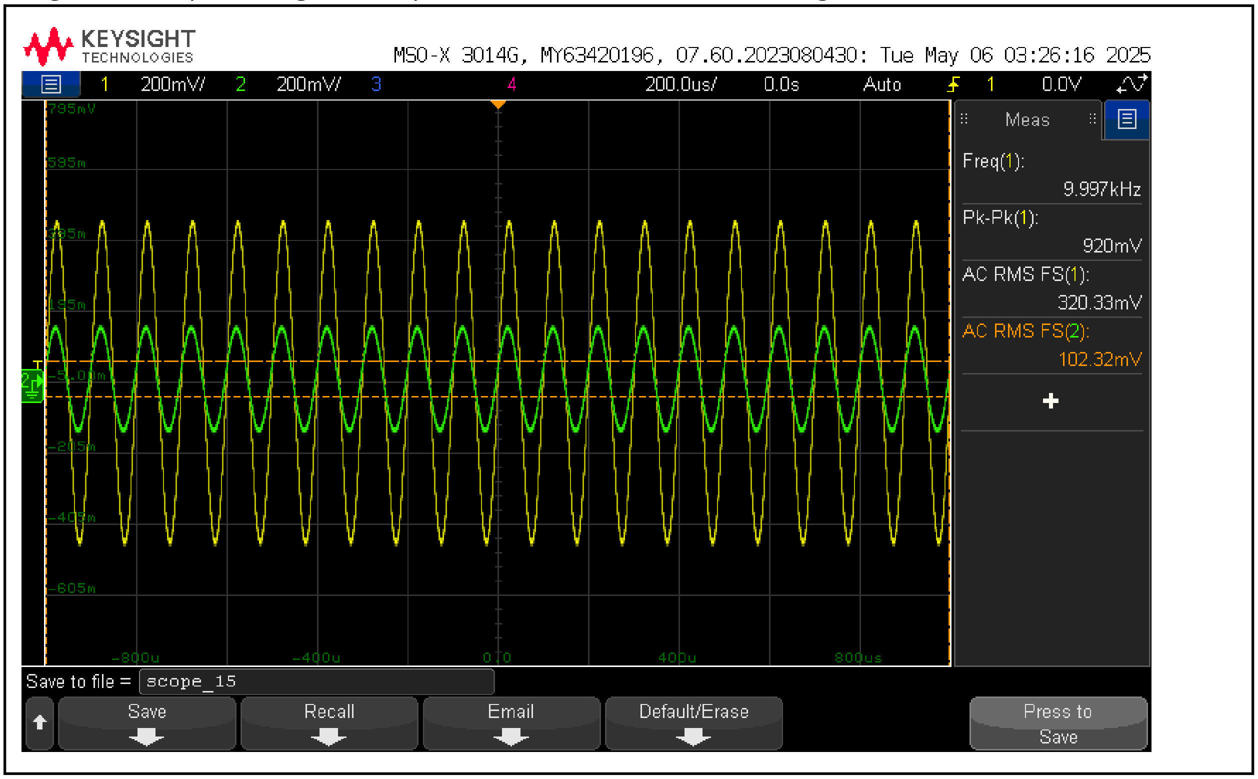


Image XVII: Output voltage with equalizer turned to maximum settings at 10kHz




**Ripple Results**

Image XVIII: Summing amplifer output FRA to calculate ripple

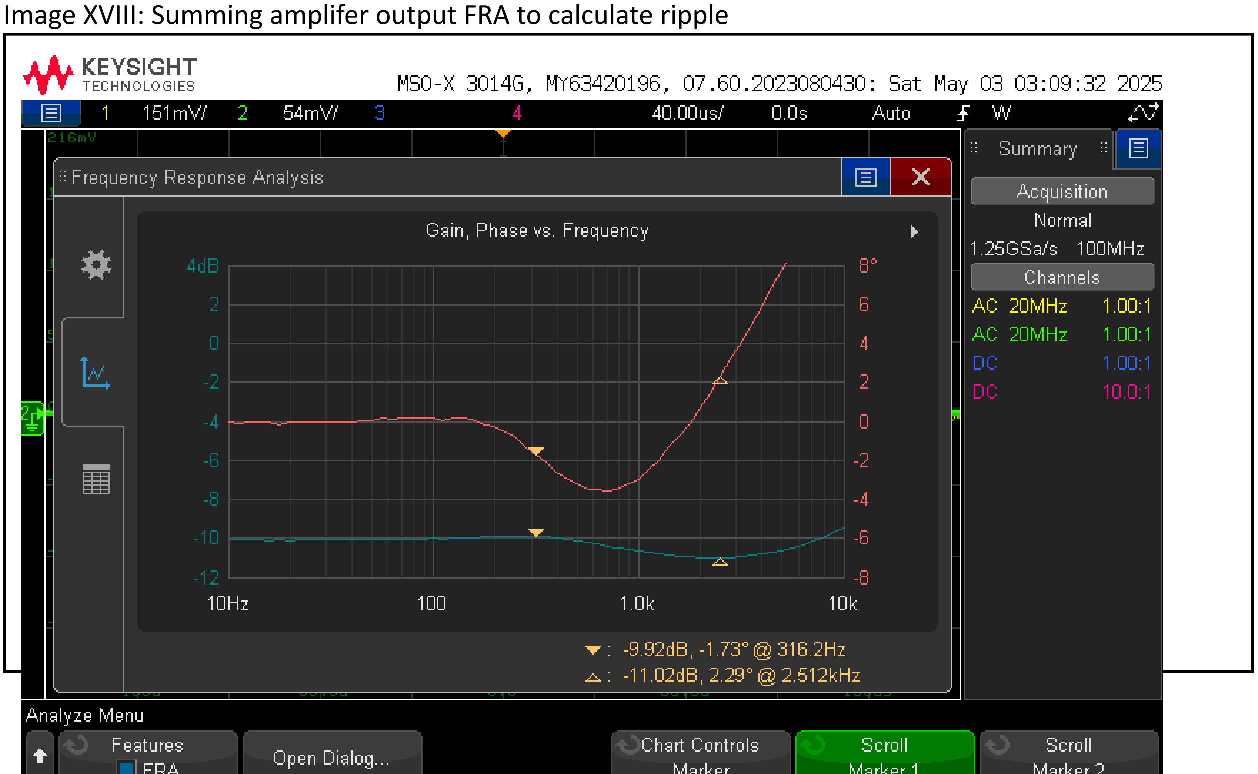


Image XIX: RMS Voltage at 316.2Hz

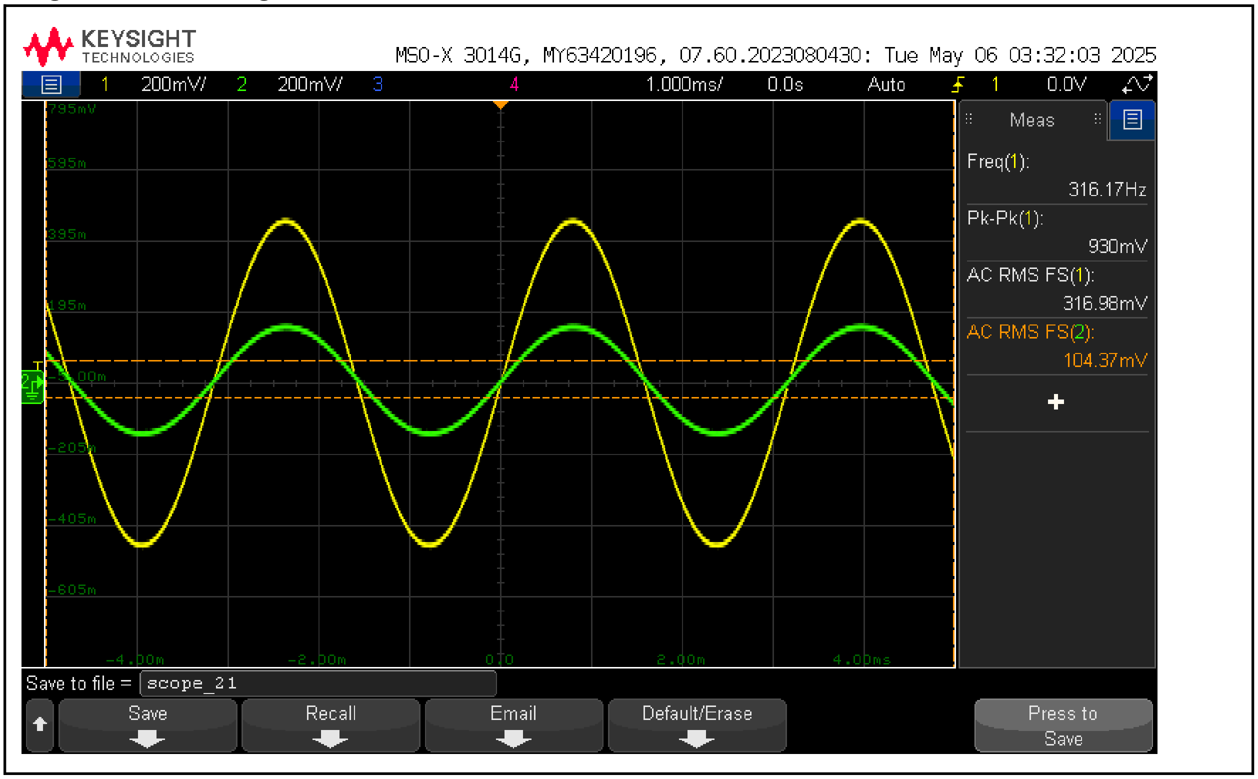
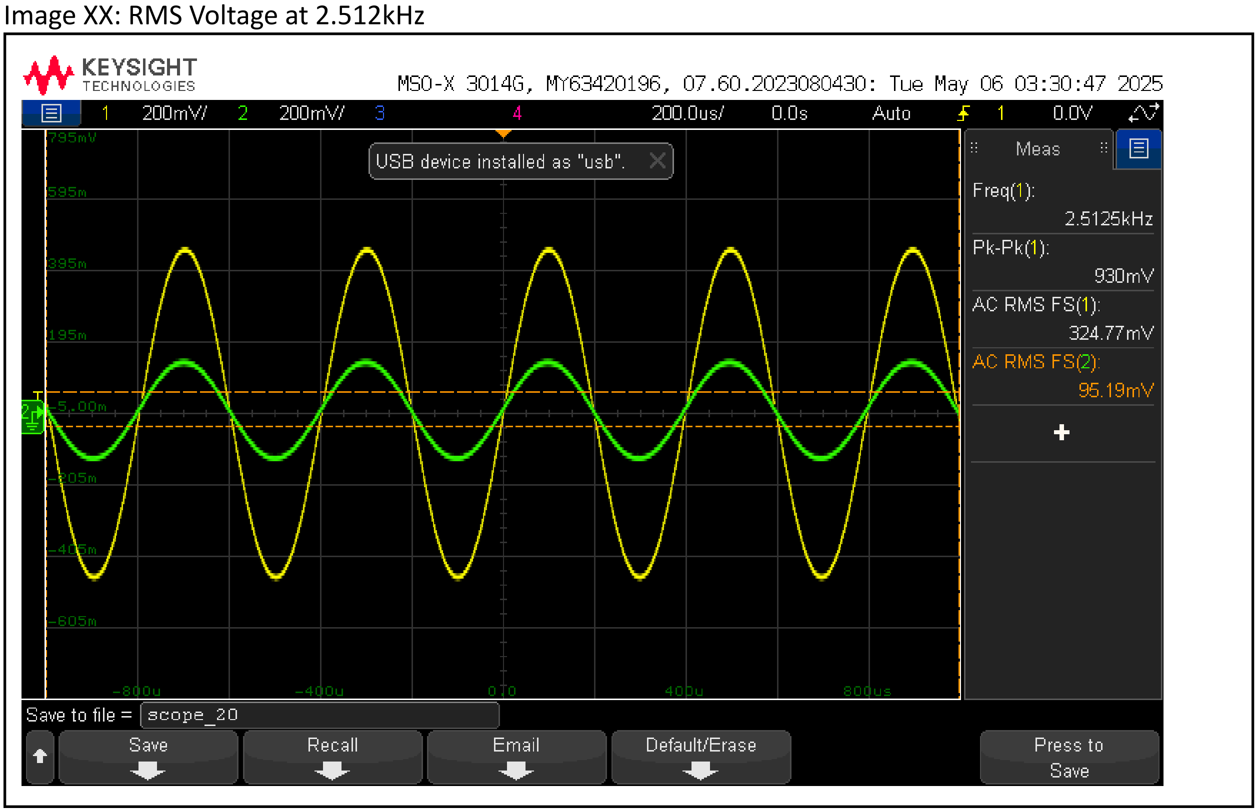



Image XX: RMS Voltage at 2.512kHz



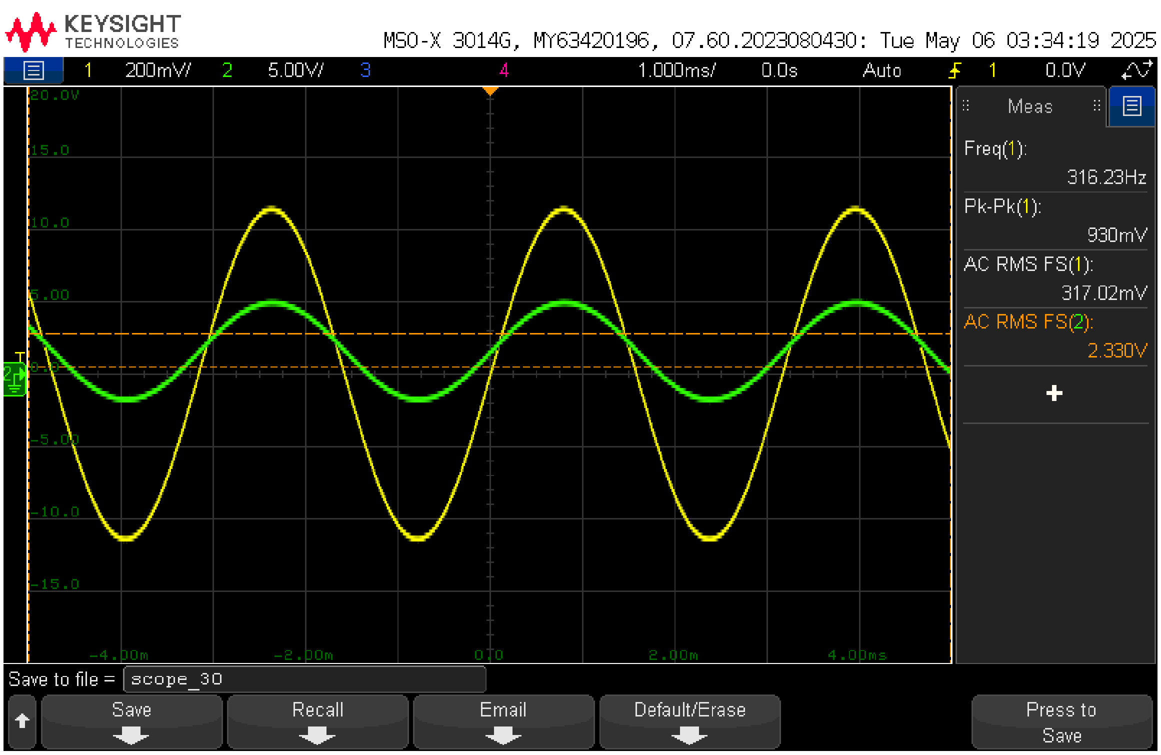
As seen in the summing amplifier output FRA in Image XVIII, the maximum deviation happens at 316.2Hz, and the minimum deviation happens at 2.512kHz. To calculate maximum ripple, we must look at the RMS voltage at each of these frequencies and find the difference.

At 316.2Hz, the summing amplifier outputs 104.37mVRMS. At 2.512kHz, the summing amplifier outputs 95.19mVRMS. This means that the largest ripple is 104.37m - 95.19m = 9.18mV.

9.18mVRMS is less than 15mVRMS.

**Power Amplifier Results**

Image XXI: RMS Voltage output of power amplifier.




As seen in Image XXI, the RMS output of the power amplifier is 2.33VRMS. We also know that the speaker has a resistance of 8 ohms.

Since $P = \frac{V^2}{R}$, we know that $P = \frac{2.33^2}{8} = 0.678$ watts.

0.678W is greater than 400mW.

**Volume Control Results**

Image XXII: Audio equalizer output at high volume

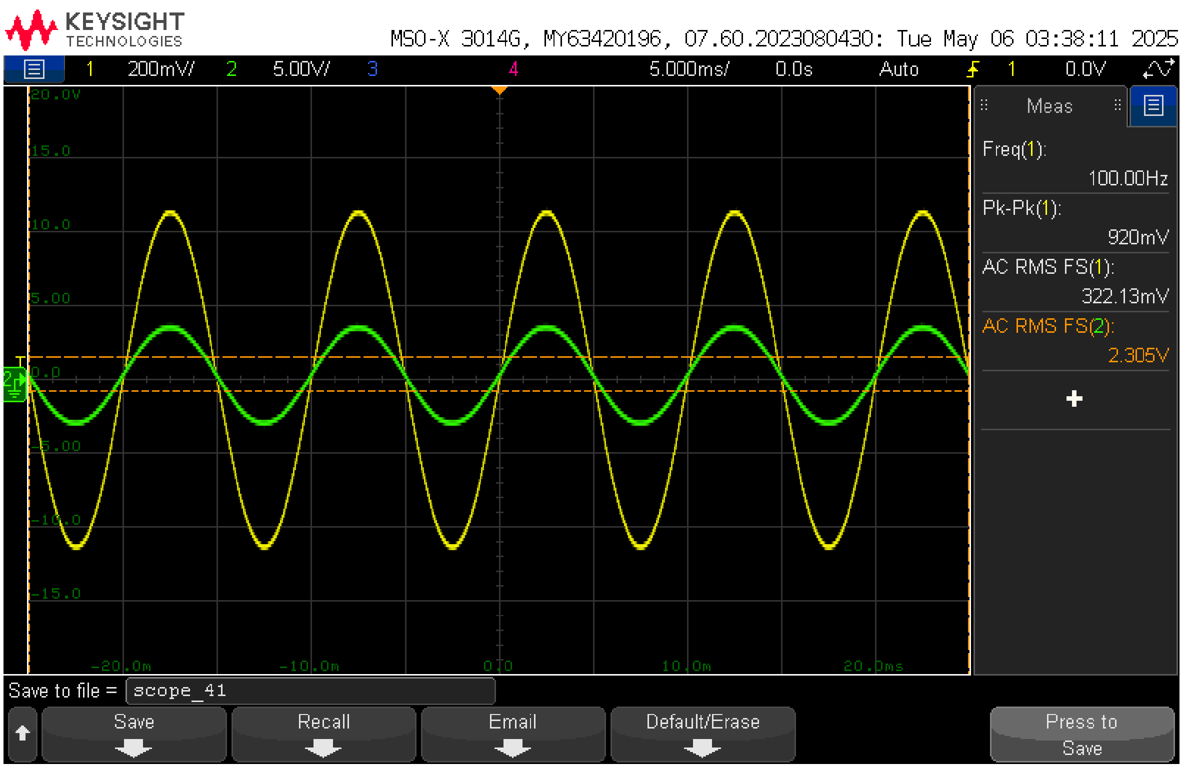



Image XXIII: Audio equalizer output at medium volume

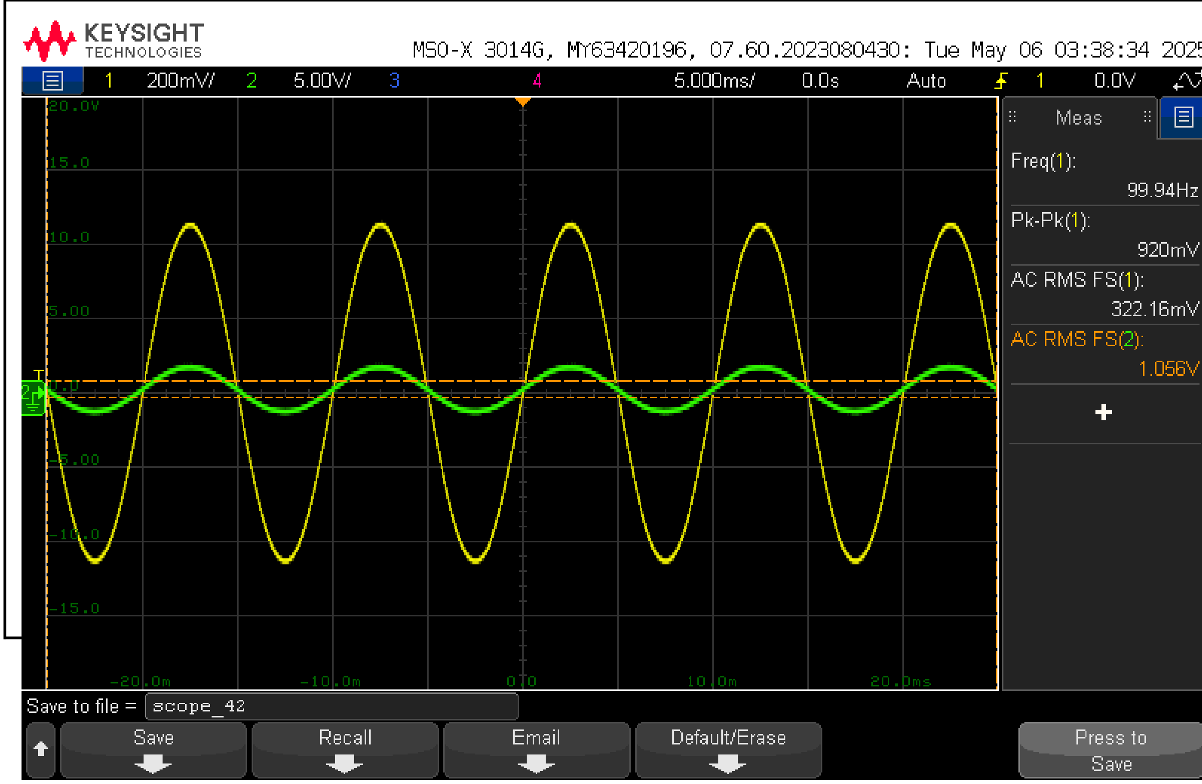
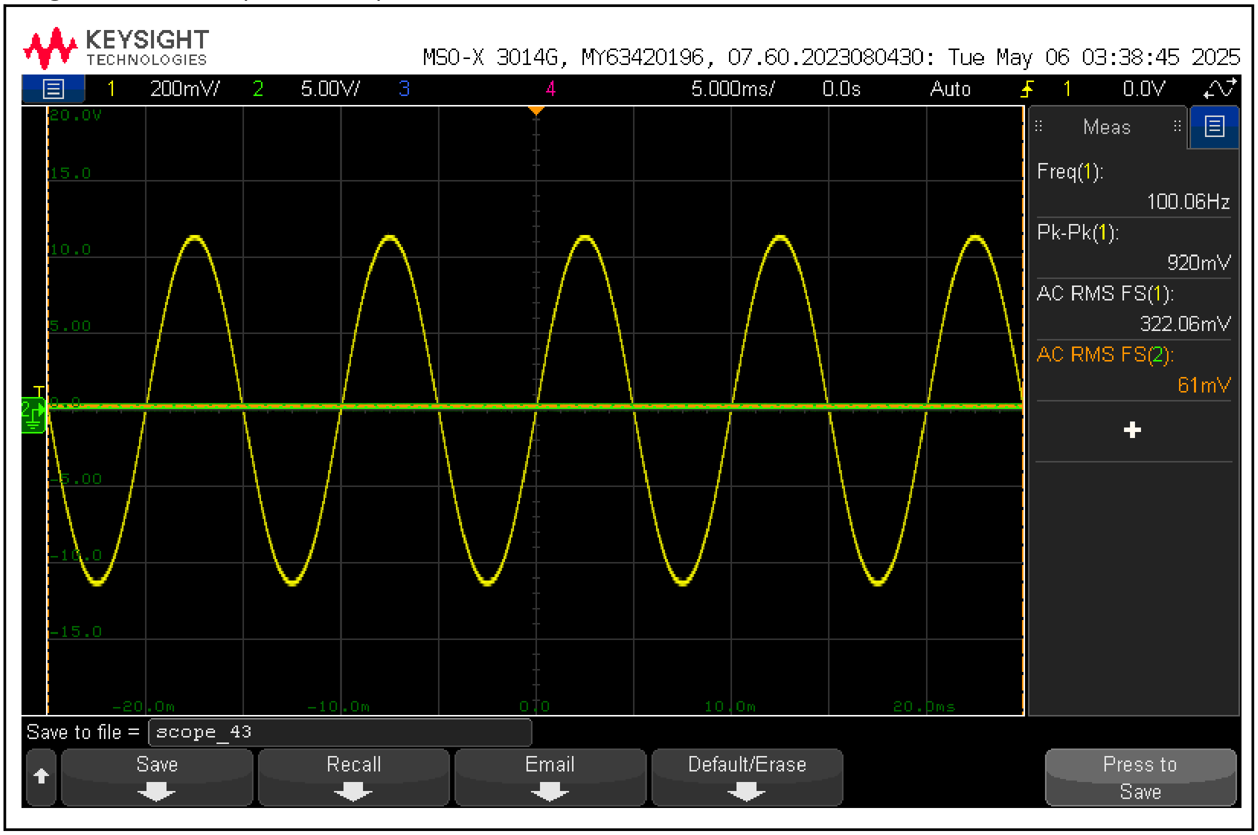


Image XXII: Audio equalizer output at low volume



**Results Justification**

Overall, all of my audio equalizer outputs met the set constraints for a functional audio equalizer and operational functions. See below for a table of the constraints and the outputs of my circuit.

Table V: Expected values, measured values, and percent error (if applicable) for each constraint

| | Expected Value | Measured Value | Percent Error |
|---|---|---|---|
| All settings low,<br>VRMS @ 100Hz | < 15mVRMS | 2.38mVRMS | n/a |
| All settings low,<br>VRMS @ 1kHz | < 15mVRMS | 2.17mVRMS | n/a |
| All settings low,<br>VRMS @ 10kHz | < 15mVRMS | 2.30mVRMS | n/a |
| All settings high,<br>VRMS @ 100Hz | 100mVRMS ± 10% | 101.86mVRMS | 1.86% |
| All settings high,<br>VRMS @ 1kHz | 100mVRMS ± 10% | 94.61mVRMS | 5.39% |
| All settings hgih,<br>VRMS @ 10kHz | 100mVRMS ± 10% | 102.32mVRMS | 2.32% |
| Ripple | < 15mRMS | 9.18mVRMS | n/a |
| Power | > 400mW | 678.61mW | n/a |

## Conclusion

Overall, this audio equalizer was a success, with functioning adjustable bass, mid, and treble gains, as well as an overall volume knob. This project took all of my knowledge of operational amplifiers, filters, and circuit design skills and combined them all into one large circuit, giving me the opportunity to integrate all of these concepts together.

The greatest sources of inaccuracy were in my filters and the loading resistors for my summing amplifier. The filters did not always hit 0dB at their peaks (as seen in the FRAs), which caused a lot of error in my loading resistors. I had to tweak the loading resistors significantly, resulting in them deviating by a lot from the calculated values of 33k. Additionally, the power amplifier amplified my signal by too much, and caused the speaker to crack when I sent an audio signal to my circuit. I toned it down by adding a small resistor, and the audio sounded great.

In the future, I could improve on this project by further adjusting the accuracy of the filters. Due to a lack of time (and finals season!) I was unable to devote the necessary testing time to constant FRAs of new filter values. Additionally, I could integrate another set of filters to send the signal out to two different speakers (a tweeter and a woofer) and further improve the audio quality. Finally, I would love to replace the band-pass filter with an RLC circuit to improve the audio quality significantly. Due to a lack of inductors, I was unable to create this RLC filter, but I could look into sourcing new inductors to make this a reality.

Thank you for reading, and I hope you enjoyed my audio equalizer project!

## References

**Works Cited**

Texas Instruments. *LM386 Low Voltage Audio Power Amplifier*. May 2004.

Texas Instruments. *LFx5x JFET Input Operational Amplifiers.* May 2000.

Texas Instruments. *LMx24, LMx24x, LMx24xx, LM2902, LM2902x, LM2902xx, LM2902xxx Quadruple Operational Amplifiers.* September 1975.